

Architecture du système 5DON4D

Singh Jashan 62727

Saeed Anees 62294

El Hichou Abderrahman
63009

Systèmes & Répartition des Données

Système	Type	Rôle Primaire	Cas d'Usage
MongoDB	Document	Source de vérité	Users, Meetings, Outbox Events
Neo4j	Graphe	Analyse relations	Recommandations, réseau social
Elasticsearch	Full-Text	Moteur recherche	Recherche users/meetings
Redis	Key-Value	Cache	Profils, scores, recherche
Kafka	Message Broker	Synchro async	Événements métier, Outbox

- **microservices**
 - **4 services NoSQL**
 - **MongoDB**
 - **ElasticSearch**
 - **Neo4j**
 - **Redis**
 - **service d'événement/data streaming**
 - **Kafka**
- **Spring Boot comme couche applicative**
- **Communication asynchrone via Kafka**
- **Pattern Outbox**

MongoDB - Stockage Principal

- modèle document
 - flexible pour les profils utilisateurs
- support natif des lists (interests, participants, ...)
- Replica Set pour le Pattern Outbox

MongoDB - Exemple de collection

```
// Collection: users
{
  "_id": "67a1b2c3d4e5f6g7h8i9j0k1",
  "name": "Alice Dupont",
  "email": "alice@example.com",
  "bio": "Passionnée de tech et running",
  "interests": ["tech", "running",
  "cooking"],
  "totalPoints": 245,
  "totalMeetings": 12
}
```

MongoDB - Exemple de collection

```
// Collection: meetings
{
  "_id": "m1a2b3c4d5e6f7g8h9i0",
  "title": "Morning Run au Bois de la Cambre",
  "eventType": "sport",
  "date": "2025-01-15T07:00:00",
  "location": "Bois de la Cambre",
  "organizer": "67a1b2c3d4e5f6g7h8i9j0k1",
  "participants": ["67a1...", "89x2...",
"45y7..."],
  "maxParticipants": 8,
  "interests": ["running", "nature"],
  "status": "UPCOMING",
  "points": 0,
  "createdAt": "2025-01-10T14:30:00"
}
```

MongoDB - Exemple de collection

```
// Collection: outbox_events
{
  "_id": "out123",
  "aggregateId":
"67a1b2c3d4e5f6g7h8i9j0k1",
  "aggregateType": "User",
  "eventType": "UserCreatedEvent",
  "payload": "{...JSON...}",
  "processed": false,
  "createdAt": "2025-01-10T14:30:00"
}
```

MongoDB - Requêtes

```
// Trouver les meetings d'un utilisateur
// Fourni par Spring Boot
meetingRepo.findByParticipantsContaining(userId)
;

// Regex case-insensitive sur interests
@Query("{ 'interests': { $elemMatch: { $regex: ?0, $options: 'i' } } }")
List<Meeting> findByInterestsRegexIgnoreCase(String interest);
```

MongoDB - Justification & Requêtes

Avancées

- // 1. Aggregation : Top 10 utilisateurs
- db.users.aggregate([
- { \$match: { totalMeetings: { \$gte: 5 } } },
- { \$sort: { totalPoints: -1 } },
- { \$limit: 10 }
-])

- // 2. Regex case-insensitive
- db.users.find({
- interests: { \$elemMatch: {
- \$regex: "tech", \$options: "i"
- } }
- })

- // 3. Spring Data Query
- @Query("{'interests': { \$elemMatch: {
- \$regex: ?0, \$options: 'i'
- } } }")
- List<Meeting> findByInterestsRegexIgnoreCase(String interest);

Neo4j - Graphe Social

- modèle graphe natif
 - pratique pour représenter les rencontres entre users
- calcul efficace des relations transitive (amis d'amis) comparé à du modèle relationnel
- score social agrégé par utilisateur
- détection de communautés et “distance sociale” dans le graphe

Neo4j - Noeud/Relation

```
// Nœud User
(:User {
  id:
  "67a1b2c3d4e5f6g7h8i9j0k1",
  name: "Alice Dupont"
})
```

```
// Relation MET
(:User)-[:MET {
  meetingId: "m1a2b3c4d5e6f7g8h9i0",
  points: 25,
  date: datetime("2025-01-
15T07:00:00"),
  location: "Bois de la Cambre",
  interest: "running, nature"
}]->(:User)
```

Neo4j - Requêtes

```
// Recommandations basées sur amis communs
MATCH (me:User {id: $userId})-[:MET]->(friend)-[:MET]-
>(recommendation:User)
WHERE NOT (me)-[:MET]-(recommendation) AND me <> recommendation
RETURN recommendation.id, recommendation.name, count(DISTINCT friend)
AS mutualFriends
ORDER BY mutualFriends DESC
```

Neo4j - Requêtes

```
// Suggestions d'amis d'amis (profondeur 5)
MATCH p = (u:User {id: $userId})-[:MET*1..5]-(connected:User)
WHERE connected.id <> $userId AND NOT (u)-[:MET]-(connected)
WITH connected, min(length(p)) AS distance
RETURN connected.id AS userId, connected.name AS userName, distance AS
distance
ORDER BY distance, userName
```

Neo4j - Cas d'Usage Détaillés

- // Recommandations (friends of friends)
- MATCH (me:User {id: \$userId})
- MATCH (me)-[:MET]->(friend)-[:MET]->(rec:User)
- WHERE NOT (me)-[:MET]-(rec) AND me <> rec
- RETURN rec.id, rec.name,
- count(DISTINCT friend) AS mutualFriends
- ORDER BY mutualFriends DESC

- // Distance sociale
- MATCH path = shortestPath(
• (alice:User {id: \$userId1})
• -[:MET*]-
• (bob:User {id: \$userId2})
•)
• RETURN length(path) AS distance

- // Score agrégé par utilisateur
- MATCH (u:User {id: \$userId})-[r:MET]->()
• RETURN u.name, SUM(r.points) AS totalScore

ElasticSearch - Recherche Avancée

- recherche tolérante aux fautes de frappe
- fuzziness AUTO : 2 marges d'erreur
- recherche sur base des caractéristiques d'un user
 - son nom
 - un de ses intérêts
- recherche sur base des intérêts
 - affiche les users ayant tous les intérêts
cherchés

Elasticsearch - Requêtes Avancées

- // 1. Fuzzy matching sur nom/bio
 - GET /users/_search
 - {
 - "query": {
 - "multi_match": {
 - "query": "alice",
 - "fields": ["name^2", "bio", "interests"],
 - "fuzziness": "AUTO"
 - }
 - }
 - }
-
- // 2. Recherche combinée (bool query)
 - {
 - "query": {
 - "bool": {
 - "should": [
 - { "match": { "name": "alice" } },
 - { "match": { "bio": "développeuse" } }
 -]
 - }
 - }
 - }

ElasticSearch - Index

// Index: users

```
{  
  "userId": "67a1b2c3d4e5f6g7h8i9j0k1",  
  "name": "Alice Dupont",  
  "email": "alice@example.com",  
  "bio": "Passionnée de tech et running",  
  "interests": ["tech", "running",  
  "cooking"],  
  "totalScore": 245,  
  "lastActive": "2025-01-10T14:30:00"  
}
```

// Index: meetings

```
{  
  "meetingId":  
  "m1a2b3c4d5e6f7g8h9i0",  
  "title": "Morning Run",  
  "location": "Bois de la Cambre",  
  "interests": ["running", "nature"],  
  "status": "upcoming",  
  "date": "2025-01-15T07:00:00",  
  "participants": ["67a1...", "89x2..."]  
}
```

Redis - Cache

- réduction des requêtes vers MongoDB et Elasticsearch
 - décharge ces DB en diminuant les requêtes doublons
- TTL adapté aux modifications
- Éviction du cache pour la cohérence des données
 - automatique sur modification (@CacheEvict)
- Clear du cache “search” à chaque mise à jour
- Granularité pour les meetings par status

Redis - Cache

Cache “user”

- TTL : 1 heure
- Clé : user::{userId}
- Valeur : User complet

Cache “interests”

- TTL : 10 minutes
- Clé : interests::all
- Valeur :
List<String>

Cache “search”

- TTL : 5 mins
- Clé : search::nameBio:{query}
- Valeur :
List<UserSearchDocument>

Cache “meetingsByStatus”

- TTL : 5 minutes
- Clé :
meetingsByStatus::{status}
- Valeur : List<Meeting>

Kafka - Messaging Asynchrone

- Découplage entre les systèmes
- Résilience
 - chaque consumer est indépendant (groupid distinct)
- Garantie de livraison avec le Pattern Outbox
 - Tolérance aux pannes
- Scalabilité horizontale via partitions

Kafka - Topics

Topic “user-events” (3 partitions)

- UserCreatedEvent → Neo4j & ElasticSearch (création noeud/index)
- UserUpdatedEvent → ElasticSearch (update index)

Topic “meeting-events” (3 partitions)

- MeetingCreatedEvent → ElasticSearch (création index)
- MeetingUpdatedEvent → ElasticSearch (update participants)
- MeetingCompletedEvent → Neo4j (création relations)
- MeetingCompletedEvent → ElasticSearch (update status)
- MeetingCancelledEvent → ElasticSearch (update status)

Kafka - Topics

Pourquoi des événements distincts ?

- Granularité fine : chaque action métier = 1 événement
- Historique : traçabilité complète du cycle de vie
- Consumers spécialisés : chaque événement déclenche des actions différentes
- Evolutivité : ajout facile de nouveaux consumers

Séparation des responsabilités :

- MeetingCreatedEvent & MeetingUpdatedEvent & MeetingCancelledEvent → Elasticsearch uniquement
- MeetingCompletedEvent → Neo4j + Elasticsearch (car création de relations sociales)

Kafka - Exemples de messages

```
{
  "topic": "meeting-events",
  "eventType": "MeetingCreatedEvent",
  "data": {
    "meetingId": "m1a2b3c4d5e6f7g8h9i0",
    "title": "Morning Run au Bois de la
Cambre",
    "eventType": "sport",
    "date": "2025-01-15T07:00:00",
    "location": "Bois de la Cambre",
    "organizer": "67a1...",
    "participants": ["67a1..."],
    "maxParticipants": 8,
    "interests": ["running", "nature"],
    "createdAt": "2025-01-10T14:30:00"
  }
}
```

```
{
  "topic": "meeting-events",
  "eventType": "MeetingCompletedEvent",
  "data": {
    "meetingId": "m1a2b3c4d5e6f7g8h9i0",
    "participants": ["67a1...", "89x2...", "45y7..."],
    "pointsPerUser": {
      "67a1...": 25,
      "89x2...": 30,
      "45y7...": 20
    },
    "date": "2025-01-15T07:00:00",
    "location": "Bois de la Cambre",
    "interests": "running, nature"
  }
}
```

Kafka - Cycle de vie d'un meeting

1. Création

- createMeeting() → MeetingCreatedEvent → Elasticsearch (index)

1. Participation

- joinMeeting() → MeetingUpdatedEvent (JOIN) → Elasticsearch (update participants)
- leaveMeeting() → MeetingUpdatedEvent (LEAVE) → Elasticsearch (update participants)

1. Finalisation

- completeMeeting() → MeetingCompletedEvent → Neo4j (relations) + Elasticsearch (status)
- cancelMeeting() → MeetingCancelledEvent → Elasticsearch (status=cancelled)

Flux de synchronisation des données

1. Api Request : création d'un user

- POST /api/register

2. Transaction MongoDB

- Save User
- Save OutboxEvent
(processed=false)

3. OutboxEventPublisher

- @Scheduled 5s : lis les OutboxEvent toutes les 5 secondes
- Fetch unprocessed events (processed=false)
- Publish to Kafka to the “user-events” topic
- Mark processed=true

4. Synchronisation vers Neo4j et ElasticSearch

- 2 Listeners Kafka du topic “user-events”
 - 2 Consumers
 - Neo4j (groupid: neo4j-consumer)
 - ElasticSearch (groupid: elasticsearch-consumer)
- Neo4j : créer le nouveau noeud pour l'utilisateur
- ElasticSearch : index UserSearchDocument créé/mis à jour

Pattern Outbox - Garantie de Cohérence

1. Écriture atomique

- Objet + OutboxEvent dans la même transaction MongoDB

2. Publication asynchrone

- Scheduler lit l'outbox toutes les 5 secondes

3. Idempotence

- Flag “processed” évite la double-publication

4. Retry automatique

- Événements non-traités restent dans l'outbox
- Ils seront réessayés en cas d'erreur d'écriture sur le consumer
- Exemple
 - Création d'un user
 - Save dans Mongo + OutboxEvent publié
 - Listener ElasticSearch save l'user mais Neo4j échoue
 - L'offset de lecture des messages du Listener Neo4j n'aura pas avancé, il reste à celui qui a échoué et sera retry automatiquement

- Aucune perte d'évènements
- Ordre des messages garantis par "createdAt"
 - date de création de l'évènement
 - donc si l'utilisateur fait rapidement
 - create
 - update 1
 - update 2
 - l'ordre est create/update1/update2
 - grâce à la date de l'évènement publié à Kafka
- Découplage entre les services
- Monitoring via le flag "processed"

```
@Transactional
public User register(RegisterRequest request) {
    User savedUser = userRepo.save(newUser);

    // Même transaction atomique
    UserCreatedEvent event = new UserCreatedEvent(...);
    outboxService.publishEvent(
        savedUser.getId(), "User", "UserCreatedEvent",
        event
    );

    return savedUser;
}
```

Kafka - Garantie d'ordre

Clé de partition : `aggregateld`

- `kafkaTemplate.send(topic, event.getAggregateld(), payload)`

Utilisation de partitions

- Parallélisme : 3 meetings différents traités simultanément
- Isolation : un meeting bloqué n'affecte pas les autres

Conséquences :

- Tous les évènements d'un même objet se retrouvent dans la même partition
- Ordre FIFO (First in First out) garanti au sein de la partition
 - Create → Update 1 → Update 2 → Update 3
- Create échoue
 - Consumer ne commit pas l'offset d'avancement dans la file
 - Kafka retry automatiquement Create
 - Updates en attente
- Cohérence des données : pas d'updates avant le create

Topic meeting-events

Meeting M1 (aggregateld="m1") → Partition

0

- CreateEvent (offset 0)
- UpdateEvent (offset 1)
- CompletedEvent (offset 2)

Meeting M2 (aggregateld="m2") → Partition

1

- CreateEvent (offset 0)
- CancelledEvent (offset 1)

Meeting M3 (aggregateld="m3") → Partition 2

- CreateEvent (offset 0) ← ÉCHEC
 - Retry automatique et Consumer bloqué
- UpdateEvent (offset 1) ← En attente

Architecture Services & Repositories

Architecture Modulaire Spring Boot

Services Métiers:

- AuthService - Authentification et inscription
- UserService - Gestion des profils utilisateurs
- MeetingService - Gestion des rencontres
- SocialGraphService - Opérations sur le graphe Neo4j
- SearchService - Recherche via Elasticsearch
- PointsCalculationService - Calcul des points

Repositories:

- UserRepository (MongoDB)
 - MeetingRepository (MongoDB)
 - UserNodeRepository (Neo4j)
 - UserSearchRepository (Elasticsearch)
 - OutboxEventRepository (MongoDB)
-
- Cache: Redis avec @Cacheable et @CacheEvict

Cycle Complet - Création Utilisateur

- Flux Détaillé : User Registration
- 1. USER REGISTRATION
POST /api/auth/register
- 2. MONGODB WRITE (Transaction ACID)
Save User + OutboxEvent (processed=false)
- 3. OUTBOX PUBLISHER (@Scheduled 5s)
Publish to Kafka topic 'user-events'
Mark processed=true
- 4. KAFKA CONSUMERS
Neo4j: CREATE (:User {id, name})
Elasticsearch: INDEX UserSearchDocument
- 5. REDIS CACHE (optionnel)
Cache user profile (1h TTL)
- 6. SYNCHRONIZATION COMPLETE

Cas d'Usage - Simuler une Rencontre

- Scénario : Alice et Bob se rencontrent
- 1. API: POST /api/meetings
- 2. MONGODB: INSERT Meeting + OutboxEvent
- 3. KAFKA → ELASTICSEARCH: Index Meeting
- 4. USER JOINS: MeetingUpdatedEvent
- 5. COMPLETE MEETING: POST /api/meetings/{id}/complete
- 6. MONGODB: UPDATE status + points
- 7. KAFKA → NEO4J: CREATE relations MET
- 8. KAFKA → ELASTICSEARCH: UPDATE status
- 9. REDIS: Invalidate cache
- 10. RECOMMANDATIONS ACTIVÉES via SocialGraphService

Merci !